

1. XPath absoluto vs relativo

1.1. XPath absoluto (evitarlo en tests)

`/html/body/div[2]/div[1]/form/input[1]`

- Depende de toda la estructura del DOM.
- Cualquier cambio en el layout lo rompe.
- En automatización es muy frágil → casi nunca conviene.

1.2. XPath relativo (lo que sí queremos)

`//form[@id='loginForm']/input[@name='username']`

- Empieza con `//` (desde cualquier parte del documento) o con `.` (relativo al elemento actual).
- Se apoya en **atributos, texto y relaciones lógicas**, no en la ruta completa.

En testing automatizado (Selenium, Playwright, Cypress + plugin XPath, etc.), casi siempre conviene **XPath relativo**.

2. Selección básica por tag y atributos

2.1. Un atributo

`//input[@id='username']`

- `//input` → cualquier `<input>`.
- `[@id='username']` → con ese `id` exacto.

Ejemplo Selenium (Java):

```
driver.findElement(By.xpath("//input[@id='username']"));
```

2.2. Múltiples atributos (con **and** / **or**)

```
//input[@type='text' and @name='username']  
//button[@type='submit' and @class='btn btn-primary']
```

También se puede con **or**:

```
//input[@name='email' or @name='userEmail']
```

2.3. Negación con **not()**

```
//input[@type='text' and not(@disabled)]  
//button[not(contains(@class, 'hidden'))]
```

3. Comodines importantes: *****, **@***, **node()**

3.1. ***** – cualquier tipo de tag

Cualquier elemento con cierto atributo

```
//*[@id='login']  
//*[@data-testid='btn-guardar']
```

- ***** = cualquier etiqueta (**div**, **span**, **button**, etc.).

Cualquier hijo directo

```
//ul[@id='menu']/*
```

- Todos los hijos directos de **ul#menu**, sin importar el tag.

Cualquier elemento con cierto atributo dentro de un contexto

```
//form[@id='loginForm']//*[@required]
```

3.2. **@*** – cualquier atributo

Algún atributo con valor exacto

```
//*[@*='login']
```

Algún atributo que contenga un texto

```
//*[contains(@*, 'login')]
```

- Útil para debugging o búsqueda rápida cuando no sabés qué atributo contiene cierto texto.
-

3.3. `node()` – cualquier nodo (texto + elementos)

```
//div[@id='mensaje']/node()
```

- Devuelve todos los nodos hijos de ese `div` (texto, spans, etc.).
 - En testing suele bastar con `*` o usar `text()`, pero `node()` existe y es bueno conocerlo.
-

4. Uso de texto: `text()`, `.` y `normalize-space()`

4.1. Texto exacto

```
//button[text()='Login']  
//a[text()='Cerrar sesión']
```

4.2. `contains(text(), ...)`

```
//button[contains(text(), 'Login')]  
//div[contains(text(), 'Operación exitosa')]
```

4.3. `normalize-space()` (ignora espacios extra)

```
//button[normalize-space(text())='Login']
```

```
//span[normalize-space(.)='Total a pagar']
```

- `normalize-space` limpia espacios al inicio, final y múltiples internos.
- `.` en vez de `text()` contempla todos los textos descendientes (por ejemplo si hay un `<span>` adentro).

Ejemplo de botón con espacios raros:

```
//button[normalize-space(.)='Iniciar sesión']
```

5. Funciones de coincidencia parcial: `contains()` y `starts-with()`

5.1. IDs / clases dinámicos con `contains()`

```
//input[contains(@id, 'user')]
//div[contains(@class, 'error-message')]
```

Ejemplo: botón con id dinámico `btn-login-123`:

```
//button[contains(@id, 'btn-login')]
```

5.2. `starts-with()`

```
//input[starts-with(@id, 'user_')]
```

- Útil para IDs generados como `user_123`, `user_456`, etc.

(En XPath 1.0 no existe `ends-with()`, así que se usa `contains()` como workaround.)

6. Contexto relativo: `.` y `..`

6.1. . como “este elemento”

Si ya tenés un elemento padre:

```
WebElement form = driver.findElement(By.id("loginForm"));
WebElement userInput = form.findElement(By.xpath("//input[@name='username']"));
```

XPath:

```
./input[@name='username']
```

- `.` → se refiere al nodo actual.
- `./` → “cualquier descendiente del nodo actual”.

6.2. .. para ir al padre

```
//input[@id='username']/..
```

- Devuelve el padre del input.
 - Equivalente a `parent::node()`.
-

7. Relaciones jerárquicas: /, // y ejes básicos

7.1. Hijo directo / y descendiente //

```
//ul[@id='menu']/li    # li hijos directos
//ul[@id='menu']/li    # li en cualquier profundidad
```

7.2. Ejes `parent::` y `ancestor::`

```
//span[text()='Total']/parent::div
//span[text()='Total']/ancestor::form
```

- **parent::** → padre directo.
- **ancestor::** → algún ancestro (padre, abuelo, etc.).

Ejemplo típico:

```
//h3[normalize-space(.)='Producto  
X']/ancestor::div[contains(@class,'card')]/button[normalize-space(.)='Comprar']
```

8. Axes útiles para testing

8.1. **following-sibling::** y **preceding-sibling::**

Label → **input** que le sigue

```
<label>Usuario</label>  
<input type="text" name="user">
```

```
//label[normalize-space(.)='Usuario']/following-sibling::input[1]
```

Label después del **input**

```
<input type="checkbox" id="accept">  
<label for="accept">Acepto los términos</label>
```

```
//label[normalize-space(.)='Acepto los términos']/preceding-sibling::input[1]
```

8.2. **following::** y **preceding::** (no solo hermanos)

- **following::** → cualquier nodo posterior en el documento.
- **preceding::** → cualquier nodo anterior.

Ejemplo: input que aparece después de un label, sin importar nivel:

```
//label[normalize-space(.)='Email']/following::input[1]
```

8.3. **child::**, **descendant::**, **self::**

```
//div[@id='card']/child::button  
//div[@id='card']/descendant::button  
//button[self::button[@type='submit']]
```

En la práctica, se usa:

- / en vez de **child::**
 - // en vez de **descendant::**
 - **self::** se ve más en XPaths generados o muy avanzados.
-

9. Índices y funciones de posición: **position()**, **last()**

9.1. Índices simples

```
(//input[@type='text'])[1] # primer input de texto  
(//button[@type='button'])[3] # tercer botón
```

Usar índices puros es **frágil**; mejor combinarlos con filtros.

9.2. **position()** dentro de un contexto

```
//ul[@id='menu']/li[position()=2]
```

9.3. **last()**

```
(//div[@class='row'])[last()  
//table[@id='users']/tr[last()]
```

- Útil para tomar la última fila de una tabla de resultados, último mensaje, etc.
-

10. Múltiples predicados y **not()**

10.1. Encadenar filtros

```
//input[@type='text'][@name='username'][not(@disabled)]
```

Equivale a:

```
//input[@type='text' and @name='username' and not(@disabled)]
```

10.2. Negar por texto

```
//button[not(normalize-space(.)='Cancelar')]
```

10.3. Atributo no vacío

```
//input[@name and @name!=""]
```

```
//input[@name and not(@name="")]
```

11. Unión de XPaths con **|** (OR de nodos)

Permite soportar varias variantes de UI con un solo locator.

```
//button[@id='login'] | //button[@data-testid='btn-login']
```

Otro ejemplo (multi-idioma):

```
//button[normalize-space(.)='Guardar'] | //button[normalize-space(.)='Save']
```

Selenium:

```
By loginBtn = By.xpath("//button[@id='login'] | //button[@data-testid='btn-login']");
```

12. Patrones típicos orientados a testing

12.1. Input relativo a un label

```
//label[normalize-space(.)='Email']/following::input[1]
```

- Útil cuando el input no tiene buenos atributos, pero el label sí.
-

12.2. Botón dentro de un formulario específico

```
//form[@id='loginForm']//button[@type='submit']
```

```
//form[@id='loginForm']//button[normalize-space(.)='Login']
```

12.3. Elemento dentro de una fila de tabla por texto

```
//table[@id='users']//tr[td[normalize-space(.)='Juan Perez']]//a[normalize-space(.)='Editar']
```

- `//tr[td[...]]` → fila que tenga un `td` con ese texto.
 - `//a[...]` → link “Editar” dentro de esa fila.
-

12.4. Mensaje de error asociado a un campo

```
//input[@name='email']/ancestor::div[contains(@class,'form-group')]//span[contains(@class,'error')]
```

12.5. Cards de producto + botón dentro

```
//div[contains(@class,'product-card')] and ./h3[normalize-space(.)='Auriculares XYZ']//button[contains(normalize-space(.),'Agregar al carrito')]
```

- Card que tenga ese título.
 - Botón con texto “Agregar al carrito” dentro de esa card.
-

12.6. Modal específico + botón dentro del modal

```
//div[@role='dialog' and ./h2[normalize-space(.)='Editar usuario']]//button[normalize-space(.)='Guardar']
```

12.7. Soportar cambios de tag (**button** ↔ **a**)

```
//*[self::button or self::a][normalize-space(.)='Login']
```

O usando roles:

```
//*[@role='button' and normalize-space(.)='Login']
```

12.8. Usar atributos especiales de testing (**data-testid**, **data-qa**)

Cuando el equipo de desarrollo lo permite, esto es lo ideal:

```
//*[@data-testid='login-button']  
//*[contains(@data-qa, 'user-row')]
```

Son **mucho más estables** que **class** o **id** en muchos proyectos modernos.

13. Buenas prácticas para XPaths relativos en automatización

1. Prioriza atributos estables:

- **data-testid**, **data-qa**, **data-role**, etc.

Evita rutas absolutas y contadas por índice:

```
//div[2]/div[1]/ul/li[3]/a
```

✗ muy frágil

2.

3. Usa texto cuando sea estable:

- Botones, links, labels estáticos.

4. Combina contexto + atributos:

- Ej. botón dentro de un formulario o card concretos, en lugar de toda la página.

5. Minimiza índices como último recurso:

- Si los usás, que sea sobre un grupo ya muy filtrado.

6. Prefiere `./` cuando ya tenés un `WebElement` padre:

- Menos posibilidades de matchear el elemento equivocado en otra sección.

7. Tené cuidado con selectores demasiado genéricos (`//*`, `//*[contains(@*, 'x')]`):

- Úsalos para debugging o como base que después refinás.

Tabla Resumen

1. Selección básica y comodines

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>//tag[@attr='valor']</code>	Tag con atributo exacto	<code>//input[@id='username']</code>	Campo con <code>id</code> fijo
<code>//tag[@a='v1' and @b='v2']</code>	Múltiples atributos	<code>//input[@type='text' and @name='username']</code>	Hacer el locator más específico
<code>//*[@attr='valor']</code>	Cualquier tag con atributo	<code>//*[@data-testid='login-btn']</code>	Usar atributos de testing (<code>data-*</code>)
<code>//*[@*='valor']</code>	Algún atributo con valor exacto	<code>//*[@*='login']</code>	Búsqueda amplia / debugging
<code>//*[contains(@*, 'valor')]</code>	Algún atributo contiene texto	<code>//*[contains(@*, 'btn-primary')]</code>	Hallar elementos cuando no sabés en qué atributo está

2. Texto del elemento

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>//tag[text()='Texto']</code>	Texto exacto	<code>//button[text()='Login']</code>	Botones/links con texto fijo
<code>//tag[contains(text(),'Texto')]</code>	Texto parcial	<code>//div[contains(text(), 'exitoso')]</code>	Mensajes de alerta/éxito
<code>//tag[normalize-space(text())='Texto']</code>	Ignora espacios extra	<code>//a[normalize-space(text())='Cerrar sesión']</code>	Cuando hay espacios raros
<code>//tag[normalize-space(.)='Texto']</code>	Texto normalizado del nodo completo	<code>//button[normalize-space(.)='Iniciar sesión']</code>	Botón con spans internos

3. Funciones de coincidencia parcial

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>//tag[contains(@attr, 'val')]</code>	Atributo contiene texto	<code>//input[contains(@id, 'user')]</code>	IDs/clases semidinámicas
<code>//tag[starts-with(@attr, 'val')]</code>	Atributo empieza con texto	<code>//div[starts-with(@id, 'user_')]</code>	IDs con sufijo aleatorio
<code>//tag[not(@attr)]</code>	Tag que no tiene atributo	<code>//input[@type='text' and not(@readonly)]</code>	Inputs editables
<code>//tag[@attr!='']</code>	Atributo no vacío	<code>//input[@name!='']</code>	Validar que haya nombre/ID

4. Jerarquía, contexto y axes básicos

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>//padre/hijo</code>	Hijo directo	<code>//ul[@id='menu']/li</code>	Items directos de un menú
<code>//padre//desc</code>	Descendiente a cualquier nivel	<code>//form[@id='loginForm']//input</code>	Inputs dentro de un form
<code>../tag[...]</code>	Desde el elemento actual	<code>../input[@name='username']</code>	Dentro de un <code>WebElement</code> padre
<code>//tag/..</code>	Padre directo	<code>//input[@id='user']/..</code>	Ir al contenido de un campo
<code>//hijo/parent::padre</code>	Padre directo con tipo	<code>//input[@id='user']/parent::div</code>	Padre con tipo explícito
<code>//tag/ancestor::padre</code>	Cualquier ancestro	<code>//span[text()='Total']/ancestor::form</code>	Encontrar el <code><form></code> de un campo

5. Axes entre hermanos y en el documento

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>//label[...]/following-sibling::input[1]</code>	Hermano siguiente	<code>//label[.='Usuario']/following-sibling::input[1]</code>	Input inmediatamente después del label
<code>//label[...]/preceding-sibling::input[1]</code>	Hermano anterior	<code>//label[.='Acepto']/preceding-sibling::input[1]</code>	Checkbox antes del label

<code>//label[...]/following::input[1]</code>	Cualquier input posterior	<code>//label[.='Email']/following::input[1]</code>	Input que podría estar más anidado
<code>//elem/child::tag</code>	Hijo directo (equiv. <code>/tag</code>)	<code>//ul[@id='menu']/child::li</code>	Más explícito, se usa poco
<code>//elem/descendant::tag</code>	Descendiente (equiv. <code>//tag</code>)	<code>//div[@id='card']/descendant::button</code>	Botón dentro de un card
<code>//button[self::button[@type='submit']]</code>	Eje <code>self::</code>	Chequear tipo/propiedades del propio nodo	Situaciones avanzadas

6. Índices y posición

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>(//tag[@cond])[1]</code>	Primer elemento que cumple cond.	<code>(//input[@type='text'])[1]</code>	Primer input de texto
<code>(//tag[@cond])[n]</code>	n-ésimo elemento	<code>(//button[@type='button'])[3]</code>	Tercer botón de tipo button
<code>//padre/hijo[position=2]</code>	Segundo hijo que cumple	<code>//ul[@id='menu']/li[position=2]</code>	Segundo ítem de menú
<code>//tag[last()]</code>	Último elemento	<code>(//div[@class='row'])[last()]</code>	Última fila/row
<code>//table//tr[last()]</code>	Última fila de tabla	<code>//table[@id='users']//tr[last()]</code>	Último registro

(Recordá: índices puros son frágiles, usarlos solo como complemento de filtros buenos.)

7. Unión, múltiple filtrado y negación

Patrón XPath	Descripción	Ejemplo típico	Uso en testing
<code>//tag[@a='1'][@b='2']</code>	Múltiples predicados en cadena	<code>//input[@type='text'][@name='user'][not(@disabled)]</code>	Locator muy específico
<code>//tag[cond1 and cond2]</code>	Condiciones combinadas	<code>//button[@type='submit' and contains(@class,'primary')]</code>	Botón principal de submit
<code>//tag[not(cond)]</code>	Negación	<code>//button[not(contains(@class,'hidden'))]</code>	Cualquier botón visible
<code>`path1`</code>	<code>`path2`</code>	Unión de nodos (OR)	<code>`//button[@id='login']`</code>
<code>//button[.='Guardar'] //button[.='Guardar']</code>	Texto alternativo	Multi-idioma: ES / EN	

8. Patrones típicos de testing (recetas)

Caso	XPath ejemplo	Comentario
Input por label	<code>//label[normalize-space()='Email']/following::input[1]</code>	Evita depender del <code>id</code> del input
Botón submit de un form	<code>//form[@id='loginForm']//button[@type='submit']</code>	Dentro del form correcto
Fila de tabla por texto	<code>//table[@id='users']//tr[td[normalize-space()='Juan Perez']]</code>	Seleccionar una fila específica
Botón dentro de esa fila	<code>//table[@id='users']//tr[td[normalize-space()='Juan Perez']]//a[normalize-space()='Editar']</code>	Botón/Link de acción de esa fila

Mensaje de error de un campo	<code>//input[@name='email']/ancestor::div[contains(@class, 'form-group')]/span[contains(@class, 'error')]</code>	Error asociado a un campo
Card de producto + botón	<code>//div[contains(@class, 'product-card')] and .//h3[normalize-space(.)='Auriculares XYZ']//button[contains(normalize-space(.), 'Agregar al carrito')]</code>	Botón dentro de la card del producto correcto
Modal + botón de guardar	<code>//div[@role='dialog'] and .//h2[normalize-space(.)='Editar usuario']//button[normalize-space(.)='Guardar']</code>	Asegura que sea el botón del modal correcto
Botón que puede ser <code><button></code> o <code><a></code>	<code>//*[self::button or self::a][normalize-space(.)='Login']</code>	UI cambiante pero texto igual
Usar atributo de testing	<code>//*[@data-testid='login-button']</code>	Lo más estable si existe <code>data-testid</code>
Cualquier requerido dentro de un form	<code>//form[@id='loginForm']//*[@required]</code>	Listar/contar campos obligatorios
